Kim Knudsen, Technical University of Denmark

Inverse Problems and Imaging (02624)

Week 8

Iterative regularization and semi-convergence

Minimize the least squares functional

$$J(x) = \|Kx - y\|^2$$

by an iterative method

$$x^{k+1} = x^k + \alpha^k d^k$$

with $d^k \in X$ and $\alpha^k > 0$.

Iterative regularization and semi-convergence

Minimize the least squares functional

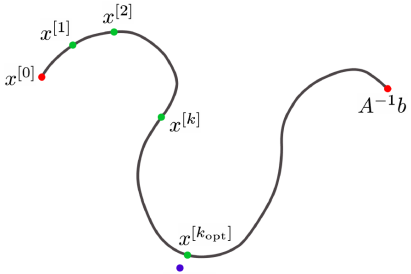
$$J(x) = \|Kx - y\|^2$$

by an iterative method

$$x^{k+1} = x^k + \alpha^k d^k$$

with $d^k \in X$ and $\alpha^k > 0$.

Stopping before convergence
is a regularization strategy
$\rightarrow$ Semiconvergence!



Landweber iteration

Steepest descent method

$$x^{k+1} = x^k - \omega K^*(Kx^k - y).$$

Converges globally if $0 < \omega < 2/\|K^*K\|$ with linear rate.

SVD analysis

$$x^k = \sum_{j=1}^{\infty} \frac{q(k,\mu_j)}{\mu_j}(y, y_j)x_j, \qquad x_0 = 0,$$

with filter factors

$$q(k,\mu_j) = (1 - (1 - \omega\mu_j^2)^k).$$

Landweber iteration

Steepest descent method

$$x^{k+1} = x^k - \omega K^*(Kx^k - y).$$

Converges globally if $0 < \omega < 2/\|K^*K\|$ with linear rate.

SVD analysis

$$x^k = \sum_{j=1}^{\infty} \frac{q(k,\mu_j)}{\mu_j}(y, y_j)x_j, \qquad x_0 = 0,$$

with filter factors

$$q(k,\mu_j) = (1 - (1 - \omega\mu_j^2)^k).$$

Slow convergence, but easy to understand, implement and analyse.

Conjugate gradient least squares

Define the Krylov subspace

$$\mathcal{K}_k = \text{span}\{K^*y, (K^*K)K^*y, \ldots, (K^*K)^k K^*y\}.$$

CGLS computes

$$x^k = \text{argmin}_{\mathcal{K}_k} \|Kx - y\|^2.$$

Faster convergence.

Algorithm for matrices

Minimize $\|Ax - y\|^2$ in $\{A^T y, (A^T A)A^T y, \ldots, (A^T A)^k A^T y\}$ by

$x^{[0]} = 0, \quad r^{[0]} = b - A\,x^{[0]}, \quad d^{[0]} = A^T r^{[0]}$

for $k = 1, 2, \ldots$

$$\bar{\alpha}_k = \frac{\|A^T r^{[k-1]}\|_2^2}{\|A\,d^{[k-1]}\|_2^2}, \quad x^{[k]} = x^{[k-1]} + \bar{\alpha}_k\,d^{[k-1]}, \quad r^{[k]} = r^{[k-1]} - \bar{\alpha}_k\,A\,d^{[k-1]}$$

$$\bar{\beta}_k = \frac{\|A^T r^{[k]}\|_2^2}{\|A^T r^{[k-1]}\|_2^2}, \quad d^{[k]} = A^T r^{[k]} + \bar{\beta}_k\,d^{[k-1]}$$

end

Implemented in Python/Matlab by `lsqr`.